COMP 2233 Data Structures — Laboratory Reference

Attachment B: C Memory Layout & String Handling Cheat Sheet

This technical cheat sheet details the internal mechanics of C string representation, pointer memory addressing, contiguous vs. non-contiguous allocation, CPU cache line prefetching, and Big-O complexity analysis required for Lab 2.

1. String Representation & Null-Terminator Mechanics

In the C programming language, strings are not primitive data types; they are represented as contiguous character arrays (char[]) terminated by a null character ('\0', ASCII value 0). Always allocate buffer size as **N + 1** to prevent memory truncation or buffer overflow.

> **Buffer Overflow Prevention**
> **Safe Input Practices:** Never use raw gets() or unbounded scanf("%s", str).
> • Use bounded scanf: scanf("%15s", name); (reads max 15 chars + appends '\0').
> • Use fgets: fgets(buffer, sizeof(buffer), stdin);

2. String Comparison Mechanics (strcmp)

In C, applying the equality operator str1 == str2 compares the **memory addresses** of the pointers, NOT string contents! To compare content, use strcmp(str1, str2) from <string.h>.

strcmp(str1, str2) Return Value	Logical Meaning	ASCII Character Comparison Logic
Return Value == 0	Exact Content Match	Both strings contain identical characters at every index up to '\0'.
Return Value < 0	str1 < str2 Lexicographically	The first non-matching char in str1 has a smaller ASCII value than str2.
Return Value > 0	str1 > str2 Lexicographically	The first non-matching char in str1 has a larger ASCII value than str2.

3. Contiguous Memory Allocation & O(1) Indexing Formula

Discrete Variables: Declaring separate variables (item0, item1, ...) causes the compiler to allocate memory slots non-contiguously across stack memory. Programmatic loop iteration is impossible.
Arrays / Struct Arrays: Arrays allocate a single continuous block of memory. The exact address of element at index *i* is computed instantly in **O(1)** time using the hardware indexing formula:

> **O(1) Random Access Indexing Math**
> **Random Access Address Formula:**
> Address(Element_i) = Base_Address + (i * sizeof(Element_Type))
>
> For char list[10][16]: Accessing index 2 computes Base + (2 * 16 bytes) = Base + 32 instantly.

4. CPU Cache Locality & Hardware Performance

Modern CPUs load data into ultra-fast L1/L2/L3 cache memory in blocks called **Cache Lines** (typically 64 bytes).
• **Spatial Locality (Cache Hit):** When an array element is accessed, adjacent contiguous elements are automatically preloaded into CPU cache, accelerating loop iterations.
• **Cache Misses:** Discrete variables scattered in memory force repeated RAM fetches, significantly degrading runtime performance.

5. Algorithm Complexity & Dataset Prerequisites

Search Algorithm	Best Case	Worst Case	Dataset Prerequisite	Hardware Memory Impact
Linear Search (Discrete)	O(1)	O(N · M)	Unsorted / Scattered	High Cache Misses, Cascading code
Linear Search (Array)	O(1)	O(N · M)	Unsorted	High Cache Hits, Iterative Loop
Binary Search (Struct Array)	O(1)	O(log N · M)	**Must be Sorted**	High Cache Hits, Divide & Conquer